PROJECT REPORT

FREE LLM CHATBOT USING PYTHON AND OLLAMA

A web-based chatbot using a locally running large language model

Submitted by: Srujana V

College: Dayananda Sagar College of Engineering

Contents

1. Introduction 3
2. Objectives 3
3. Software and System Requirements 3
4. Project Structure 3
5. System Architecture 4
6. Methodology and Workflow 4
7. Main Modules 4
7.1 Chat Interface 4
7.2 FastAPI Backend 5
7.3 Conversation Context 5
7.4 Temperature Control 5
7.5 Health-Check API 6
8. Testing and Results 6
8.1 Project Status Summary 6
8.2 Test Cases 7
8.3 Output Screenshots 7
9. How to Run the Project 8
10. Limitations 9
11. Future Enhancements 9
12. Conclusion 10
References 10

1. Introduction

Large language models (LLMs) can generate text, answer questions, and support conversational applications. This project demonstrates how to connect a local LLM to a web application without using a paid OpenAI, ChatGPT, or Gemini API. FastAPI handles HTTP requests, HTML and JavaScript provide the user interface, and Ollama runs the model locally. The project is an educational prototype for learning basic LLM integration.

Cloud-based AI services normally need an internet connection, a user account and, in many cases, payment for every request. Running the model locally with Ollama removes these requirements: prompts stay on the user's own computer, no API key is needed, and the chatbot can be used offline once the model has been downloaded.

In this project a browser-based chat page sends each message to a FastAPI backend. The backend passes the message and recent conversation context to the Mistral model running in Ollama and returns the generated answer to the page. The page also provides a temperature slider and a health-check address for confirming that the service is running. This report describes the design, the implementation, the tests carried out and their results, supported by screenshots of the working application.

2. Objectives

The main objectives of this project are:

- Develop a simple chatbot interface that runs in a web browser.
- Connect a Python backend to a language model through Ollama.
- Send user prompts to the model and display generated answers.
- Maintain recent conversation context for follow-up questions.
- Provide a temperature control in the interface.
- Provide a health endpoint to check application status and model configuration.
- Test the chatbot and record its behaviour with screenshots of the working application.

3. Software and System Requirements

Component	Purpose
Operating system	Windows
Programming language	Python 3.12
Backend framework	FastAPI
ASGI server	Uvicorn
Local LLM runtime	Ollama
Model used	Mistral
Frontend	HTML, CSS and JavaScript
Browser	Modern web browser

4. Project Structure

```
LLM-Prompt Project/
├── app.py                - FastAPI backend and LLM integration
```

```
├── requirements.txt       - Required Python packages
├── README.md              - Setup and usage instructions
└── templates/
    └── index.html         - Chatbot webpage
```

The venv or .venv folder contains the local Python environment. It normally does not need to be included in the submission ZIP because the environment can be recreated from requirements.txt.

5. System Architecture

The system contains three main parts:

- Frontend: The webpage displays the conversation, message input, Send button, and temperature slider.
- Backend: FastAPI receives chat requests, prepares the prompt and recent history, calls the configured model backend, and returns a response.
- LLM runtime: Ollama runs the Mistral model locally and generates a response using the prompt and conversation context.

User → Chat webpage → FastAPI backend → Ollama / Mistral → FastAPI → Chat webpage

6. Methodology and Workflow

1. Start Ollama and ensure the Mistral model is downloaded.
2. Activate the project's Python virtual environment, if used.
3. Start the FastAPI application.
4. Open the local web address in a browser, such as http://127.0.0.1:8000.
5. Enter a question and choose a temperature value.
6. The frontend sends the message and temperature to the backend.
7. The backend uses the configured LLM and recent conversation context to generate a reply.
8. The reply is displayed in the conversation window.
9. Open the /health endpoint separately to check application status.

7. Main Modules

7.1 Chat Interface

The webpage allows users to type a message and click Send. User prompts and assistant replies appear in the same conversation area. The interface displays the selected model and includes a temperature slider.

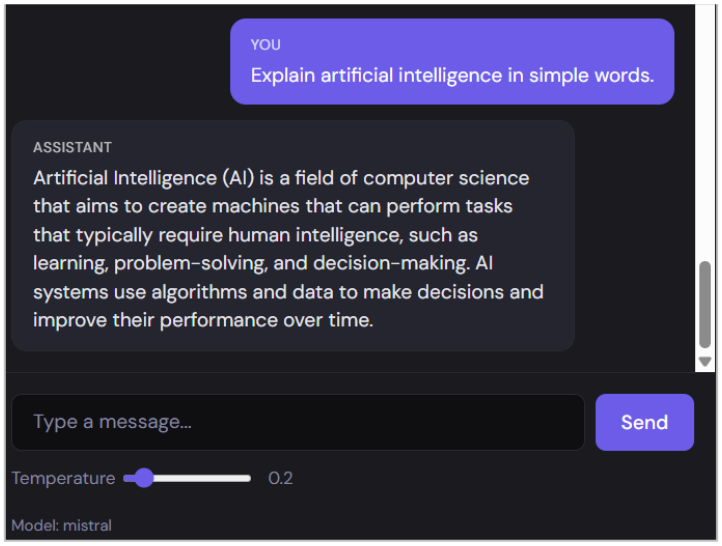


Figure 1: *Chat interface showing a question and answer, the Temperature slider set to 0.2 and the model label "mistral".*

7.2 FastAPI Backend

The backend serves the webpage and provides API routes for chat and health checking. The chat endpoint accepts a message and optional conversation history, then returns a generated reply and model information.

7.3 Conversation Context

Recent messages are supplied as context to help answer follow-up questions. In the demonstrated test, the user stated "My name is Rahul" and then asked "What is my name?" The chatbot replied that the name was Rahul. This is short-term context retention, not permanent memory; restarting the application may clear the history.

In this test the model first introduced itself as an assistant named Rahul after the user gave their name, but it then answered the follow-up question correctly. A system prompt that defines the assistant's role can reduce this kind of confusion.

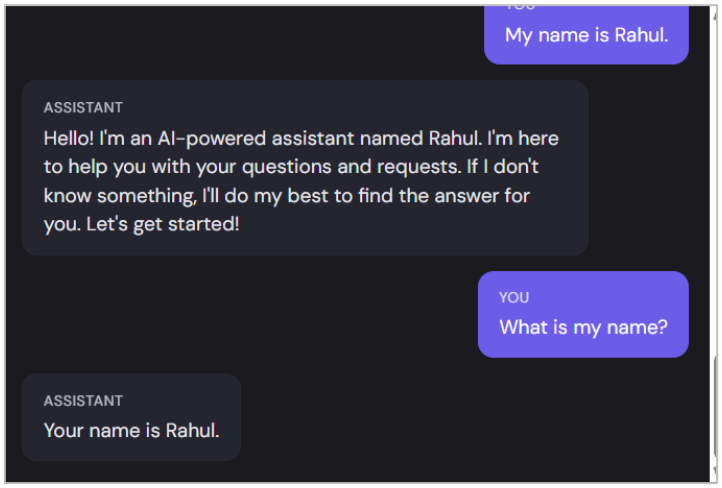


Figure 2: *Conversation-context test: after "My name is Rahul.", the chatbot answers "Your name is Rahul." to "What is my name?".*

7.4 Temperature Control

Temperature influences the variation in generated text. Lower values generally produce more predictable wording, while higher values can produce more varied wording. The selected value must be passed to the model by the backend to affect generation.

7.5 Health-Check API

The GET /health endpoint returns a JSON status response. During testing, the browser displayed {"status":"ok","backend":"ollama","model":"mistral"}. This indicates that the application reported its status as OK and identified Ollama and Mistral as the configured backend and model.

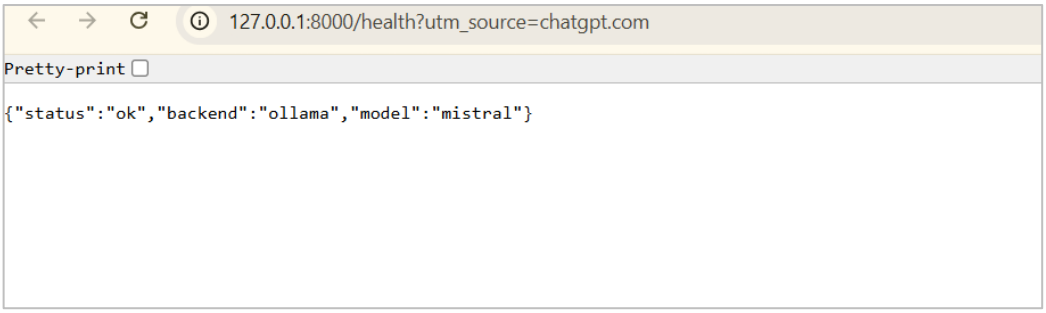


Figure 3: *Health endpoint (/health) response shown as raw JSON in the browser.*

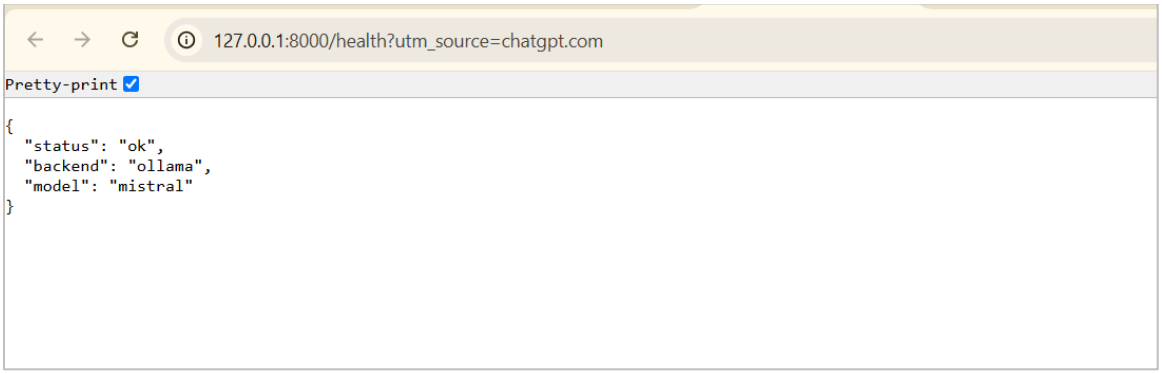


Figure 4: *Health endpoint response with browser pretty-print enabled: status "ok", backend "ollama", model "mistral".*

8. Testing and Results

8.1 Project Status Summary

All main components of the chatbot were checked. The table below summarises their status.

Project component	Status
Chatbot user interface	✓ Working
Python + FastAPI backend	✓ Working
Ollama local AI integration	✓ Working
Mistral model responses	✓ Working
Conversation memory	✓ Tested successfully
API health check	✓ Passed
Temperature control	✓ Tested at 0.2 (Figure 1)

8.2 Test Cases

Test	Procedure	Observed result
Chat response	Asked: "What is Artificial Intelligence? Explain with an example."	The chatbot generated an explanation of AI and an example involving a self-driving car.
Conversation context	Sent "My name is Rahul.", then asked "What is my name?"	The chatbot replied: "Your name is Rahul."
Health endpoint	Opened the /health URL in the browser	Returned JSON with status "ok", backend "ollama", and model "mistral".
Temperature control	Move the slider from 0.7 to 0.2 and ask: "Explain artificial intelligence in simple words."	The chatbot returned a short, direct definition of AI (Figure 1). Answers at 0.7 were longer and more detailed (Figures 5 and 7).
Knowledge question	Asked: "What is the difference between AI and machine learning?"	The chatbot explained that machine learning is a subset of AI that learns from data (Figure 6).
Structured answer	Asked: "Explain the working of a communication system in simple words."	The chatbot gave a three-step explanation: encoding, transmission and decoding (Figure 7).

The tests above were carried out on the local installation, and the screenshots in this report show the observed output. The conversation-context and health-check results appear in Section 7.

8.3 Output Screenshots

The following screenshots show further responses from the running chatbot.

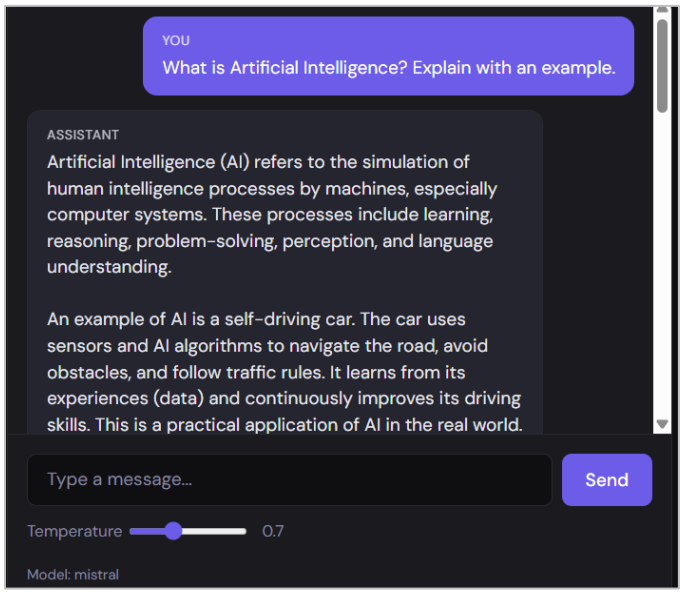


Figure 5: *Chat response at temperature 0.7: "What is Artificial Intelligence? Explain with an example."*

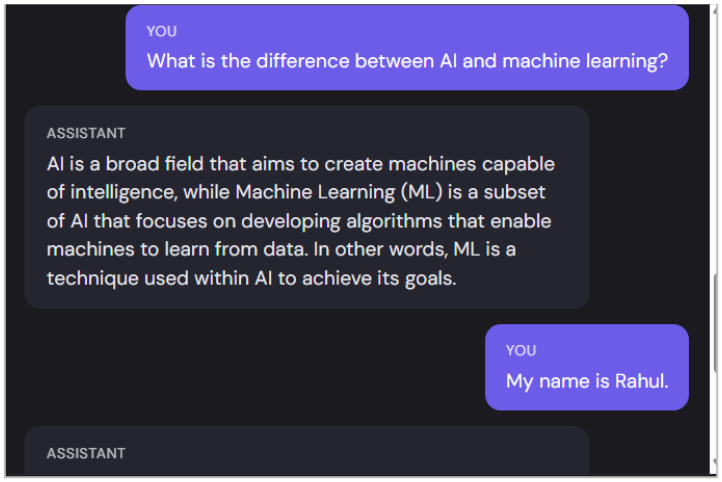


Figure 6: *Chat response to a follow-up knowledge question: the difference between AI and machine learning.*

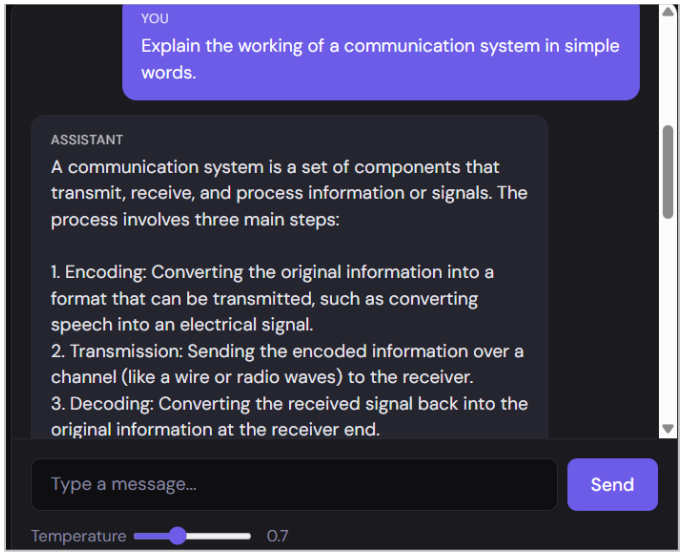


Figure 7: *Chat response at temperature 0.7 with a structured, numbered explanation of a communication system.*

9. How to Run the Project

1. Install Python and Ollama for Windows if they are not already installed.
2. Open Command Prompt or PowerShell and change directory to the project folder.
3. Install the packages listed in requirements.txt using the project's Python environment.
4. Ensure the Mistral model is available in Ollama. The first download requires an internet connection; local inference can then run on the computer.
5. Start the application with the correct environment interpreter, for example: .venv\Scripts\python.exe app.py (use venv instead if that is the folder name).
6. Keep the terminal open while using the chatbot. Open http://127.0.0.1:8000 in the browser.
7. Open http://127.0.0.1:8000/health to check the status. If the terminal shows another port, use that port instead.

Troubleshooting: if the terminal reports "only one usage of each socket address" (Errno 10048), another process is already using port 8000. Stop the earlier server, or start the application on another port, for example .venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8001, and then open http://127.0.0.1:8001.

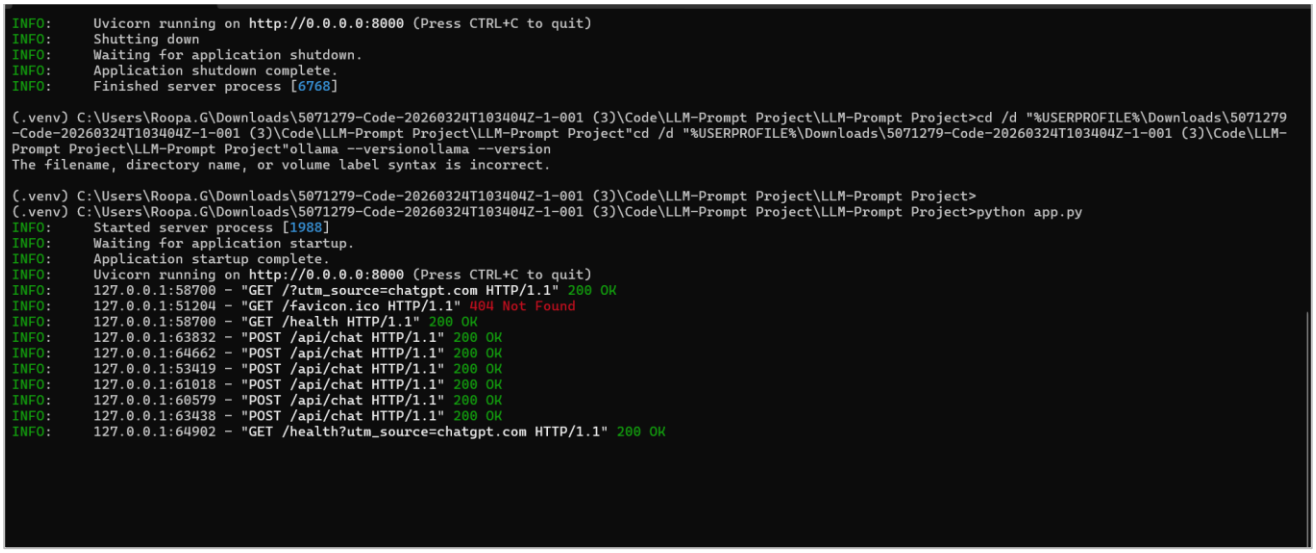


Figure 8: *Uvicorn console output: the server starts and serves the chat page, /health and /api/chat requests with 200 OK.*

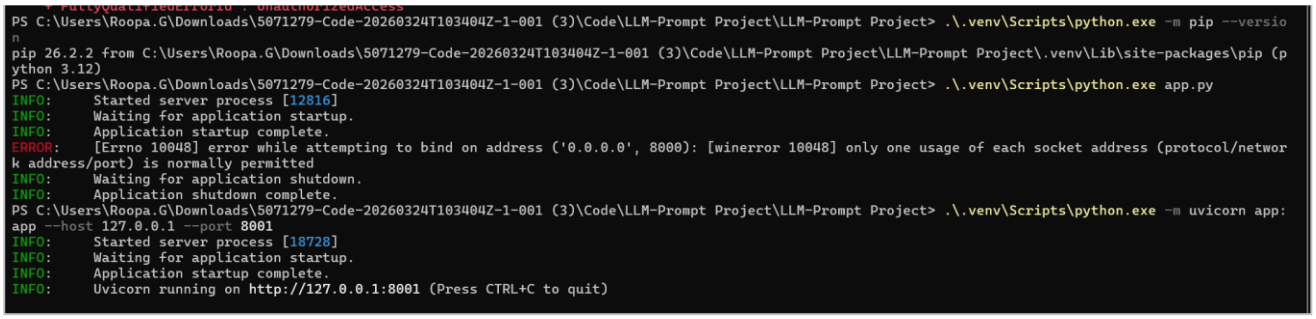


Figure 9: *Console output showing the "port already in use" error on port 8000 and a successful restart on port 8001.*

10. Limitations

- Response quality depends on the selected model and the computer's available memory and processing power.
- Conversation history is limited and may be cleared when the server or page session is restarted, depending on the implementation.
- The model may occasionally produce inaccurate information; important answers should be verified.
- The model must be downloaded before use, and the download may require several gigabytes of storage.
- This is a learning prototype and does not include production-level authentication, persistent storage, or high availability.
- Short messages can be misread: in one test the model introduced itself as an assistant named Rahul before correctly answering the follow-up question (Figure 2).

11. Future Enhancements

- Store conversation history in a database.
- Add controls to start a new conversation and clear history.
- Improve error messages when Ollama is stopped or the selected model is missing.
- Allow selection among multiple locally installed models.
- Add document question-answering or file upload.

- Improve mobile layout and accessibility.

12. Conclusion

The Free LLM Chatbot project demonstrates the integration of a local large language model with a Python web backend and a browser-based interface. The chatbot generated an answer to a general question, used recent conversation context to recall a name, and returned a healthy status from its API endpoint. Every component listed in the project status summary was tested successfully, and the screenshots in this report document the results. The project provides a practical starting point for learning prompt handling, local LLM deployment, web APIs, and chatbot interface development.

References

1. FastAPI documentation: https://fastapi.tiangolo.com/
2. Ollama documentation: https://docs.ollama.com/
3. Python documentation: https://docs.python.org/3/
4. Uvicorn documentation: https://www.uvicorn.org/